TRY / EXCEPT LEARNINGS -- CONTINUED FROM 2026-08-07

try / except -- What I Learned, part 2

Picks up right after step 7 in try_except_learnings_2026-08-07.pdf -- same file, same ingest_dimensions.py, one new requirement: stop the function if a read comes back with zero rows.

08 Logging is not stopping -- the guard that didn't guard

First attempt at the zero-row check:

```
if row_count == 0:
    logger.exception("There are no rows in the file %s", file_name)
```

This logs a message and then falls straight through -- there's no raise, no return. Execution continues past the if block, finishes the try, and reaches the write step. The empty DataFrame gets written to bronze anyway, exactly as if row_count had been checked and everything was fine.

Second problem, separate from the missing raise: logger.exception() is meant to be called from inside an except block -- it pulls the traceback of whatever exception is currently being handled. Called here, with nothing actually caught, there is no exception in flight, so the traceback it tries to attach is empty or misleading. logger.error(...) is the right call when you're flagging a bad state yourself, not reporting a caught exception.

09 Bare raise only works inside except

The fix needs an actual raise after logger.error(...) -- but a bare raise (no arguments) only means something specific: "re-raise whatever exception is currently being handled." Inside a plain if block, with nothing caught, there's nothing to re-raise -- a bare raise there would itself error out. What's needed instead is raising something new, with its own message:

```
if row_count == 0:
    logger.error("There are no rows in the file %s", file_name)
    raise ValueError(f"No rows read for {file_name}")
```

WHAT HAPPENS NEXT

This raise is still textually inside the outer try block, just nested one level deeper inside the if. Python doesn't care about the nesting -- it looks for the nearest enclosing except that matches. ValueError is an Exception, so the existing except Exception: catches it, logs "Failed to read %s" plus the full traceback (which includes the ValueError's own message), and re-raises. That propagates out of ingest_dimensions() into main()'s loop, which has no try/except of its own -- so the whole run stops there, and none of the remaining tables get processed.

10 Proved it with a minimal repro instead of just reasoning about it

Stripped the same shape down to a standalone script and ran it twice, once with row_count=5 and once with row_count=0:

```
def process(row_count):
    try:
        logger.info("Read %d rows", row_count)
        if row_count == 0:
            logger.error("There are no rows")
            raise ValueError(f"No rows read (row_count={row_count})")
    except Exception:
        logger.exception("Failed to read")
        raise
```

```
    print("writing now...")  # only reached if nothing was raised above
```

```
row_count=5 -->  INFO Read 5 rows
                  writing now...

row_count=0 -->  INFO Read 0 rows
                  ERROR There are no rows
                  ERROR Failed to read
                  Traceback (most recent call last):
                    ...
                  ValueError: No rows read (row_count=0)
                  (no "writing now..." -- the function actually stopped)
```

"writing now..." never printing for row_count=0 is the actual proof the fix works -- not the log lines, which would print either way if the raise were missing. When testing a stop-the-function fix, check for the absence of what comes after, not just the presence of the error message.

11 Side lesson: don't paste multi-line snippets into the plain REPL

While testing the above, pasting the function into python3's interactive shell (>>>) produced IndentationError: unexpected indent -- twice, for two different reasons, neither one a bug in the code:

First: pasting just the print("writing now...") line alone, with no def/try/if before it in that paste. The REPL reads line by line -- with nothing open to indent into, an indented line is an error.

Second: pasting the def block, then the print(...) line separately, with a blank line in between. In the REPL, a blank line always means "this block is done" -- even if more indented lines were meant to follow. The function closed early, and the print line arrived as an orphaned, still-indented statement.

> **LESSON**
> Run multi-line snippets as a script file (python3 script.py) instead of pasting them into the interactive shell line by line. It sidesteps this entire class of REPL-parsing quirks, which have nothing to do with whether the code itself is correct.

New rules to add to the list

- Logging something is not the same as stopping the function. Check for a raise (or return), not just a log line, when you need execution to actually halt.
- A bare raise only works inside an except block, where it means "re-raise what I just caught." Anywhere else -- a plain if, ordinary code -- raise something with its own message instead.
- logger.exception() belongs inside except, where there's a real exception to attach. Outside of one, use logger.error() or logger.warning().
- A raise nested inside an if, inside a try, still gets caught by that try's except -- Python doesn't care how deep the nesting is, only whether it's inside the try.
- When verifying a fix that's supposed to stop something, check for the absence of what comes after -- not just the presence of an error message, which can print either way.